

ANN

Experiment No. 1 ANN

Title

To Demonstrate Commonly Used Activation Functions in Neural Networks

Aim

To study and demonstrate different activation functions used in Artificial Neural Networks such as Sigmoid, ReLU, Leaky ReLU, and Softmax using Python.

Objectives

1. To understand the importance of activation functions in neural networks.
 2. To implement commonly used activation functions using Python.
 3. To visualize the behavior of activation functions using graphs.
 4. To compare outputs of different activation functions.
-

Theory

Activation functions are mathematical functions used in neural networks to determine the output of neurons. They introduce non-linearity into the network, enabling it to learn complex patterns and relationships in data.

Without activation functions, the neural network behaves like a simple linear regression model and cannot solve complex problems such as image recognition, speech processing, and classification tasks.

Activation functions help neural networks:

- Learn non-linear patterns
- Improve prediction capability
- Decide whether a neuron should be activated
- Convert input signals into meaningful outputs

Different activation functions are used depending on the type of problem and neural network architecture.

Activation Functions Used

1) Sigmoid Function

The Sigmoid function converts input values into values between 0 and 1.

Formula

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Features

- Output range: 0 to 1
- Used in binary classification
- Smooth and differentiable

Limitation

- Suffers from vanishing gradient problem
-

2) ReLU (Rectified Linear Unit)

ReLU outputs the input directly if positive; otherwise, it returns 0.

Formula

$$f(x) = \max(0, x)$$

Features

- Simple and fast
- Most commonly used activation function
- Reduces vanishing gradient problem

Limitation

- Can cause dead neurons for negative inputs
-

3) Leaky ReLU

Leaky ReLU is an improved version of ReLU that allows small negative values.

Formula

$$f(x) = \begin{cases} x, & x > 0 \\ 0.1x, & x \leq 0 \end{cases}$$

Features

- Prevents dead neuron problem
 - Better learning for negative inputs
-

4) Softmax Function

Softmax converts input values into probability distributions.

Formula

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Features

- Output values sum to 1
 - Used in multi-class classification
-

Software Requirements

- Python
 - NumPy Library
 - Matplotlib Library
-

Program Code

```
#import libraries
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Input range
```

```
x = np.linspace(-5, 5, 100)
```

```
# Activation functions
```

```
sigmoid = 1 / (1 + np.exp(-x))    # Sigmoid
```

```
relu = np.maximum(0, x)           # ReLU
```

```
leaky_relu = np.where(x > 0, x, 0.1*x) # Leaky ReLU
```

```
# Softmax (applied to a vector)
```

```
def softmax(z):
```

```
    exp_z = np.exp(z - np.max(z)) # stability trick
```

```
    return exp_z / exp_z.sum()
```

```
# Example input vector for softmax
```

```
z = np.array([1, 2, 3, 4, 5])
```

```
softmax_vals = softmax(z)
```

```
# Create subplots
```

```
fig, axs = plt.subplots(2, 2, figsize=(10,8))
```

```
# Plot Sigmoid
```

```
axs[0,0].plot(x, sigmoid, color='blue')
```

```
axs[0,0].set_title("Sigmoid")
```

```
axs[0,0].set_xlabel("Input")
```

```
axs[0,0].set_ylabel("Output")
```

```
# Plot ReLU
```

```
axs[0,1].plot(x, relu, color='green')
```

```
axs[0,1].set_title("ReLU")
```

```
axs[0,1].set_xlabel("Input")
```

```
axs[0,1].set_ylabel("Output")
```

```
# Plot Leaky ReLU
```

```
axs[1,0].plot(x, leaky_relu, color='red')
```

```
axs[1,0].set_title("Leaky ReLU")
axs[1,0].set_xlabel("Input")
axs[1,0].set_ylabel("Output")

# Plot Softmax (bar chart)
axs[1,1].bar(range(len(z)), softmax_vals, color='purple')
axs[1,1].set_title("Softmax")
axs[1,1].set_xlabel("Input Index")
axs[1,1].set_ylabel("Probability")

plt.tight_layout()
plt.show()
```

Explanation of Program

1. NumPy and Matplotlib libraries are imported.
2. Input values are generated from -5 to 5.
3. Sigmoid, ReLU, and Leaky ReLU activation functions are implemented.
4. A separate Softmax function is created.
5. Graphs for all activation functions are plotted using Matplotlib.
6. Softmax output is shown using a bar chart.

Step-by-Step Explanation

Step 1: Import Libraries

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

Imports NumPy for calculations and Matplotlib for plotting graphs.

Step 2: Create Input Values

```
x = np.linspace(-5, 5, 100)
```

Generates 100 values between -5 and 5.

Step 3: Sigmoid Function

```
sigmoid = 1 / (1 + np.exp(-x))
```

Calculates Sigmoid activation values between 0 and 1.

Step 4: ReLU Function

```
relu = np.maximum(0, x)
```

Converts negative values to 0 and keeps positive values unchanged.

Step 5: Leaky ReLU Function

```
leaky_relu = np.where(x > 0, x, 0.1*x)
```

Keeps positive values same and scales negative values by 0.1.

Step 6: Define Softmax Function

```
def softmax(z):
```

Defines the Softmax activation function.

Step 7: Calculate Exponential Values

```
exp_z = np.exp(z - np.max(z))
```

Calculates exponential values using a stability trick.

Step 8: Return Softmax Probabilities

```
return exp_z / exp_z.sum()
```

Converts values into probabilities whose sum equals 1.

Step 9: Create Input Vector

```
z = np.array([1, 2, 3, 4, 5])
```

Creates sample input values for Softmax.

Step 10: Compute Softmax Output

```
softmax_vals = softmax(z)
```

Calculates Softmax probabilities.

Step 11: Create Subplots

```
fig, axs = plt.subplots(2, 2, figsize=(10,8))
```

Creates a 2×2 layout for plotting graphs.

Step 12: Plot Sigmoid Graph

```
axs[0,0].plot(x, sigmoid, color='blue')
```

Plots the Sigmoid activation graph.

Step 13: Plot ReLU Graph

```
axs[0,1].plot(x, relu, color='green')
```

Plots the ReLU activation graph.

Step 14: Plot Leaky ReLU Graph

```
axs[1,0].plot(x, leaky_relu, color='red')
```

Plots the Leaky ReLU activation graph.

Step 15: Plot Softmax Graph

```
axs[1,1].bar(range(len(z)), softmax_vals, color='purple')
```

Displays Softmax probabilities using a bar chart.

Step 16: Adjust Layout

```
plt.tight_layout()
```

Adjusts spacing between graphs.

Step 17: Display Graphs

```
plt.show()
```

Displays all activation function graphs.

Output

The program displays graphs of:

- Sigmoid
- ReLU
- Leaky ReLU
- Softmax

These activation functions are commonly used in Deep Learning and Neural Networks.

Output

The program displays:

- Sigmoid curve
 - ReLU graph
 - Leaky ReLU graph
 - Softmax probability distribution graph
-

Result

Thus, commonly used activation functions in neural networks such as Sigmoid, ReLU, Leaky ReLU, and Softmax were successfully implemented and demonstrated using Python.

Conclusion

Activation functions are essential in neural networks because they help models learn complex and non-linear patterns. Different activation functions have different advantages and are selected according to application requirements. This practical demonstrates the working and graphical representation of commonly used activation functions.

Viva Questions with Answers

Q1. What is an activation function?

Answer:

An activation function is a mathematical function that determines the output of a neuron in a neural network.

Q2. Why are activation functions used?

Answer:

Activation functions introduce non-linearity into neural networks, helping them learn complex patterns.

Q3. What is the output range of the Sigmoid function?

Answer:

The output range is between 0 and 1.

Q4. Which activation function is most commonly used in deep learning?

Answer:

ReLU is the most commonly used activation function.

Q5. What is the main use of Softmax?

Answer:

Softmax is used for multi-class classification problems.

Q6. What is the advantage of Leaky ReLU over ReLU?

Answer:

Leaky ReLU prevents the dead neuron problem by allowing small negative outputs.

Experiment No. 2 ANN

Title

To Implement XOR Function Using a Backpropagation Neural Network

Aim

To implement the XOR logical function using a Backpropagation Neural Network in Python.

Objectives

1. To understand the working of Backpropagation Neural Networks.
 2. To implement XOR operation using a neural network.
 3. To train a neural network using supervised learning.
 4. To predict outputs using the trained model.
-

Theory

Artificial Neural Networks (ANNs) are computing models inspired by the human brain. A neural network consists of interconnected neurons organized into layers such as input layer, hidden layer, and output layer.

Backpropagation is one of the most important learning algorithms used in neural networks. It is a supervised learning technique in which the error generated at the output layer is propagated backward through the network to update weights and improve accuracy.

The XOR (Exclusive OR) problem is a classic example used to demonstrate the power of neural networks. XOR produces output:

- 1 when both inputs are different
- 0 when both inputs are same

The XOR function cannot be solved using a single-layer perceptron because it is a non-linear problem. Therefore, a multilayer neural network with hidden layers and backpropagation learning is required.

XOR Truth Table

Input A Input B Output

0	0	0
0	1	1
1	0	1
1	1	0

Backpropagation Algorithm Steps

1. Initialize weights randomly.
 2. Provide training inputs to the network.
 3. Calculate output using activation function.
 4. Compare predicted output with actual output.
 5. Calculate error.
 6. Propagate error backward through the network.
 7. Update weights to minimize error.
 8. Repeat until minimum error is achieved.
-

Neural Network Architecture

The neural network used in this practical contains:

- Input Layer → 2 neurons
- Hidden Layer → 2 neurons
- Output Layer → 1 neuron

The sigmoid activation function is used for training the model.

Sigmoid Activation Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The sigmoid function converts input values into outputs between 0 and 1.

Software Requirements

- Python
 - NumPy Library
 - Scikit-learn Library
-

Program Code

```
# Import necessary libraries
```

```
import numpy as np
```

```
from sklearn.neural_network import MLPClassifier # Multi-Layer Perceptron  
(feedforward NN)
```

```
# Define input dataset (XOR truth table inputs)
```

```
X = [[0,0],[0,1],[1,0],[1,1]]
```

```
# Define expected outputs (XOR truth table outputs)
```

```
# XOR outputs: 0 if inputs are same, 1 if inputs are different
```

```
y = [0,1,1,0]
```

```
# Initialize the neural network
```

```
# hidden_layer_sizes=(2,) → one hidden layer with 2 neurons
```

```
# max_iter=1000 → maximum training iterations
```

```
# activation='logistic' → sigmoid activation function
```

```
mlp = MLPClassifier(hidden_layer_sizes=(2,), max_iter=1000, activation='logistic')
```

```
# Train the model on the XOR dataset
```

```
mlp.fit(X,y)
```

```
# Predict outputs for the same inputs
```

```
print("Predictions:", mlp.predict(X))
```

Explanation of Program

1. NumPy and MLPClassifier libraries are imported.
2. XOR input dataset is defined using four input combinations.
3. Expected XOR outputs are stored in variable y.
4. A neural network model is created using MLPClassifier.
5. One hidden layer with two neurons is used.
6. Sigmoid activation function is selected.
7. The model is trained using the XOR dataset.
8. Predictions are generated for the input values.

Step-by-Step Explanation

Step 1: Import Libraries

```
import numpy as np
```

```
from sklearn.neural_network import MLPClassifier
```

Imports NumPy and MLPClassifier for creating the neural network.

Step 2: Define Input Dataset

```
X = [[0,0],[0,1],[1,0],[1,1]]
```

Creates the XOR input dataset with all possible input combinations.

Step 3: Define Output Dataset

```
y = [0,1,1,0]
```

Defines expected XOR outputs for each input pair.

Step 4: Initialize Neural Network

```
mlp = MLPClassifier(hidden_layer_sizes=(2,), max_iter=1000, activation='logistic')
```

Creates a neural network with one hidden layer containing 2 neurons.

Step 5: Train the Model

```
mlp.fit(X,y)
```

Trains the neural network using the XOR dataset.

Step 6: Predict Outputs

```
print("Predictions:", mlp.predict(X))
```

Predicts outputs for the given inputs and displays results.

Output

Predictions: [0 1 1 0]

The neural network learns the XOR logic and predicts correct outputs.

Output

Predictions: [0 1 1 0]

The predicted outputs match the XOR truth table.

Result

Thus, the XOR function was successfully implemented using a Backpropagation Neural Network in Python.

Conclusion

The XOR problem demonstrates the importance of hidden layers and backpropagation in neural networks. A single-layer perceptron cannot solve XOR because it is a non-linear problem. By using a multilayer neural network with backpropagation and sigmoid activation function, the XOR function can be successfully implemented and predicted.

Viva Questions with Answers

Q1. What is Backpropagation?

Answer:

Backpropagation is a supervised learning algorithm used to train neural networks by propagating errors backward to update weights.

Q2. What is XOR operation?

Answer:

XOR gives output 1 when inputs are different and 0 when inputs are same.

Q3. Why is XOR called a non-linear problem?

Answer:

Because its outputs cannot be separated using a single straight line.

Q4. What is the use of hidden layers?

Answer:

Hidden layers help neural networks solve complex and non-linear problems.

Q5. Which activation function is used in this practical?

Answer:

Sigmoid (logistic) activation function is used.

Q6. What is MLPClassifier?

Answer:

MLPClassifier is a Multi-Layer Perceptron classifier provided by Scikit-learn for implementing neural networks.

Experiment No. 3 ANN

Title

To Demonstrate Feedforward Neural Network with Backpropagation

Aim

To demonstrate the working of a Feedforward Neural Network using the Backpropagation learning algorithm in Python.

Objectives

1. To understand the concept of Feedforward Neural Networks.
 2. To study the Backpropagation learning algorithm.
 3. To implement a Feedforward Neural Network using Python.
 4. To train and test the neural network model.
-

Theory

A Feedforward Neural Network (FNN) is one of the simplest types of Artificial Neural Networks in which information moves only in one direction, from input layer to output layer. There are no cycles or feedback connections in the network.

A Feedforward Neural Network generally consists of:

- Input Layer
- Hidden Layer(s)
- Output Layer

Each neuron in one layer is connected to neurons in the next layer through weights.

The Backpropagation algorithm is used for training the network. During training:

1. Input data is passed forward through the network.
2. Output is generated.
3. Error is calculated by comparing predicted and actual outputs.
4. The error is propagated backward.
5. Weights are updated to reduce the error.

This process is repeated until the network achieves better accuracy.

Feedforward Neural Networks are widely used in:

- Classification
 - Pattern Recognition
 - Image Processing
 - Data Prediction
 - Speech Recognition
-

Architecture of Feedforward Neural Network

The network used in this practical contains:

- Input Layer → 3 neurons
 - Hidden Layers → 2 hidden layers with 5 neurons each
 - Output Layer → 1 neuron
-

Backpropagation Formula

Weight update in backpropagation:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial E}{\partial W}$$

Where:

- (W) = Weight
 - (η) = Learning rate
 - (E) = Error function
-

Sigmoid Activation Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The activation function helps introduce non-linearity into the network.

Software Requirements

- Python

- NumPy Library
 - Scikit-learn Library
-

Program Code

```
from sklearn.neural_network import MLPClassifier
import numpy as np

X = np.random.rand(100,3)
y = np.random.randint(0,2,100)

mlp = MLPClassifier(hidden_layer_sizes=(5,5), max_iter=1000)

mlp.fit(X,y)

print("Accuracy:", mlp.score(X,y))
```

Explanation of Program

1. Required libraries such as NumPy and MLPClassifier are imported.
2. Random input dataset with 100 samples and 3 features is generated.
3. Random binary target outputs are created.
4. A Feedforward Neural Network model is initialized.
5. Two hidden layers with 5 neurons each are used.
6. The model is trained using the fit() function.
7. Accuracy of the model is calculated using the score() function.

Step-by-Step Explanation

Step 1: Import Libraries

```
from sklearn.neural_network import MLPClassifier
```

```
import numpy as np
```

Imports MLPClassifier for creating the neural network and NumPy for numerical operations.

Step 2: Generate Input Data

```
X = np.random.rand(100, 3)
```

Generates 100 random samples with 3 features each.

Step 3: Generate Output Labels

```
y = np.random.randint(0, 2, 100)
```

Creates 100 random binary target labels containing 0 or 1.

Step 4: Initialize MLP Model

```
mlp = MLPClassifier(hidden_layer_sizes=(5, 5), max_iter=1000)
```

Creates a neural network with two hidden layers having 5 neurons each.

Step 5: Train the Model

```
mlp.fit(X, y)
```

Trains the neural network using the generated dataset.

Step 6: Calculate Accuracy

```
print("Accuracy:", mlp.score(X, y))
```

Calculates and displays the model accuracy.

Output

Accuracy: 0.85

The MLP model learns patterns from the dataset and predicts outputs with good accuracy.

Output

Accuracy: 0.85

(Note: Accuracy value may change during every execution because random data is used.)

Result

Thus, the Feedforward Neural Network with Backpropagation was successfully implemented and demonstrated using Python.

Conclusion

Feedforward Neural Networks are important models in Artificial Intelligence and Machine Learning. The Backpropagation algorithm helps the network learn by reducing prediction error through weight updates. This practical demonstrates how neural networks can be trained and evaluated using Python and Scikit-learn.

Viva Questions with Answers

Q1. What is a Feedforward Neural Network?

Answer:

A Feedforward Neural Network is a neural network in which data flows only in one direction from input layer to output layer.

Q2. What is Backpropagation?

Answer:

Backpropagation is a learning algorithm used to train neural networks by propagating error backward and updating weights.

Q3. What is the role of hidden layers?

Answer:

Hidden layers help the network learn complex and non-linear patterns.

Q4. What is MLPClassifier?**Answer:**

MLPClassifier is a Multi-Layer Perceptron classifier provided by Scikit-learn for implementing neural networks.

Q5. Why is activation function used?**Answer:**

Activation functions introduce non-linearity into the neural network.

Q6. Which library is used for neural networks in this practical?**Answer:**

Scikit-learn library is used for implementing the neural network.

Experiment No. 4 ANN

Title

To Demonstrate ART Clustering

Aim

To demonstrate clustering using ART (Adaptive Resonance Theory) concept in Python.

Objectives

1. To understand the concept of ART clustering.
 2. To study unsupervised learning techniques.
 3. To implement clustering using Python.
 4. To group similar data points into clusters.
-

Theory

ART (Adaptive Resonance Theory) is a type of neural network developed for unsupervised learning and pattern recognition. It is mainly used for clustering and classification of data.

ART networks are designed to solve the stability-plasticity problem:

- **Stability** means retaining previously learned information.
- **Plasticity** means learning new patterns without forgetting old ones.

In clustering, similar data points are grouped together into clusters based on their characteristics.

In this practical, clustering is demonstrated using Agglomerative Clustering, which is a hierarchical clustering technique related to unsupervised learning concepts.

Types of Learning in Neural Networks

1. Supervised Learning
2. Unsupervised Learning
3. Reinforcement Learning

ART clustering belongs to **Unsupervised Learning** because it does not require target outputs.

Agglomerative Clustering

Agglomerative Clustering is a bottom-up hierarchical clustering method where:

- Each data point starts as a separate cluster.
 - Similar clusters are merged step by step.
 - The process continues until the required number of clusters is formed.
-

Clustering Distance Formula

The Euclidean distance is commonly used to measure similarity between data points.

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Where:

- (x_i) and (y_i) are data points
 - (d) is the distance between points
-

Features of ART Clustering

- Performs unsupervised learning
 - Learns new patterns dynamically
 - Groups similar data automatically
 - Useful for pattern recognition and data analysis
-

Applications of ART Clustering

- Image Segmentation
- Pattern Recognition
- Medical Data Analysis
- Customer Segmentation
- Data Mining

Software Requirements

- Python
 - NumPy Library
 - Scikit-learn Library
-

Program Code

```
# Import necessary libraries

import numpy as np

from sklearn.cluster import AgglomerativeClustering # For hierarchical clustering


# Generate random dataset of 10 points in 2D space
X = np.random.rand(10,2)


# Initialize Agglomerative Clustering model
# n_clusters=2 → we want to group the data into 2 clusters
model = AgglomerativeClustering(n_clusters=2)


# Fit the model and predict cluster labels for each point
labels = model.fit_predict(X)


# Print the cluster assignments
print("Cluster labels:", labels)
```

Explanation of Program

1. NumPy and AgglomerativeClustering libraries are imported.
2. Random two-dimensional dataset is generated.
3. Agglomerative Clustering model is initialized with 2 clusters.

4. The model is trained using the dataset.
5. Cluster labels are predicted for each data point.
6. Final cluster assignments are displayed.

Step-by-Step Explanation

Step 1: Import Libraries

```
import numpy as np
```

```
from sklearn.cluster import AgglomerativeClustering
```

Imports NumPy for numerical arrays and AgglomerativeClustering for hierarchical clustering.

Step 2: Generate Dataset

```
X = np.random.rand(10,2)
```

Creates 10 random data points with 2 features each.

Step 3: Initialize Clustering Model

```
model = AgglomerativeClustering(n_clusters=2)
```

Creates a hierarchical clustering model with 2 clusters.

Step 4: Fit and Predict Clusters

```
labels = model.fit_predict(X)
```

Trains the model and assigns cluster labels to each point.

Step 5: Print Cluster Labels

```
print("Cluster labels:", labels)
```

Displays the cluster assigned to every data point.

Output

Cluster labels: [0 1 0 1 1 0 0 1 0 1]

The algorithm groups similar data points into clusters using hierarchical clustering.

Output

Cluster labels: [0 1 1 0 0 1 0 1 1 0]

(Note: Output may change during every execution because random data is used.)

Result

Thus, ART clustering was successfully demonstrated using Python.

Conclusion

ART clustering is an important unsupervised learning technique used for grouping similar data points into clusters. It helps neural networks learn new information while preserving previously learned patterns. In this practical, clustering was successfully implemented using Agglomerative Clustering in Python.

Viva Questions with Answers

Q1. What is ART in neural networks?

Answer:

ART stands for Adaptive Resonance Theory, which is used for unsupervised learning and clustering.

Q2. What is clustering?

Answer:

Clustering is the process of grouping similar data points together.

Q3. Which type of learning is used in ART clustering?

Answer:

Unsupervised learning is used in ART clustering.

Q4. What is Agglomerative Clustering?**Answer:**

Agglomerative Clustering is a hierarchical clustering technique that merges similar clusters step by step.

Q5. Which library is used for clustering in this practical?**Answer:**

Scikit-learn library is used.

Q6. What is the purpose of clustering?**Answer:**

Clustering helps in identifying similar groups or patterns in data.

Experiment No. 5 ANN

Title

To Demonstrate AND, OR, NOT, XOR, XNOR Logic Gates

Aim

To demonstrate the working of AND, OR, NOT, XOR, and XNOR logic gates using Python.

Objectives

1. To understand the concept of logic gates.
 2. To study the working of basic and universal logic operations.
 3. To generate truth tables for different logic gates.
 4. To implement logic gate operations using Python.
-

Theory

Logic gates are the basic building blocks of digital electronics and computer systems. They perform logical operations on binary inputs (0 and 1) and produce binary outputs.

Logic gates are widely used in:

- Digital circuits
- Computers
- Arithmetic operations
- Neural networks
- Control systems

Each logic gate performs a specific logical operation.

Types of Logic Gates

1) AND Gate

The AND gate produces output 1 only when both inputs are 1.

Boolean Expression

$$Y=A\cdot B$$

Truth Table

A B Output

0 0 0

0 1 0

1 0 0

1 1 1

2) OR Gate

The OR gate produces output 1 if at least one input is 1.

Boolean Expression

$$Y=A+B$$

Truth Table

A B Output

0 0 0

0 1 1

1 0 1

1 1 1

3) NOT Gate

The NOT gate inverts the input value.

Boolean Expression

$$Y=\overline{A}$$

Truth Table

A Output

0 1

A Output

1 0

4) XOR Gate

The XOR (Exclusive OR) gate produces output 1 when inputs are different.

Boolean Expression

$$Y = A \oplus B$$

Truth Table

A B Output

0 0 0

0 1 1

1 0 1

1 1 0

5) XNOR Gate

The XNOR gate produces output 1 when inputs are same.

Boolean Expression

$$Y = \overline{A \oplus B}$$

Truth Table

A B Output

0 0 1

0 1 0

1 0 0

1 1 1

Applications of Logic Gates

- Digital Circuits
 - Computer Processors
 - Arithmetic Circuits
 - Memory Devices
 - Communication Systems
-

Software Requirements

- Python
 - Pandas Library
-

Program Code

```
# Import pandas for creating a DataFrame
import pandas as pd

# Create an empty list to store rows of the truth table
data = []

# Loop through all possible values of inputs A and B (0 or 1)
for a in [0,1]:
    for b in [0,1]:
        # Append a row with results of different logic operations
        data.append([
            a,          # Input A
            b,          # Input B
            a and b,    # AND gate output
            a or b,     # OR gate output
            not a,      # NOT gate output (applied to A)
            a ^ b,      # XOR gate output (exclusive OR)
```



```
        not (a ^ b)      # XNOR gate output (inverse of XOR)
    ])

# Convert the list of rows into a DataFrame with column names
df = pd.DataFrame(data, columns=["A","B","AND","OR","NOT A","XOR","XNOR"])

# Print the truth table
print(df)
```

Explanation of Program

1. Pandas library is imported for creating a table.
2. An empty list is created to store truth table rows.
3. Nested loops generate all possible combinations of inputs A and B.
4. Logic operations AND, OR, NOT, XOR, and XNOR are performed.
5. Results are stored in a DataFrame.
6. The complete truth table is displayed as output.

Step-by-Step Explanation

Step 1: Import Library

```
import pandas as pd
```

Imports Pandas library for creating and displaying tables.

Step 2: Create Empty List

```
data = []
```

Creates an empty list to store truth table rows.

Step 3: Loop Through Input Values

```
for a in [0,1]:
```

```
for b in [0,1]:
```

Loops through all possible combinations of A and B.

Step 4: Store Logic Gate Outputs

```
data.append([
```

Adds logic gate results into the list.

Step 5: AND Operation

```
a and b
```

Returns 1 only when both inputs are 1.

Step 6: OR Operation

```
a or b
```

Returns 1 if at least one input is 1.

Step 7: NOT Operation

```
not a
```

Reverses the value of input A.

Step 8: XOR Operation

```
a ^ b
```

Returns 1 when inputs are different.

Step 9: XNOR Operation

```
not (a ^ b)
```

Returns 1 when inputs are the same.

Step 10: Create DataFrame

```
df = pd.DataFrame(data, columns=["A","B","AND","OR","NOT A","XOR","XNOR"])
```

Creates a table with column names.

Step 11: Print Truth Table

```
print(df)
```

Displays the truth table.

Output

	A	B	AND	OR	NOT A	XOR	XNOR
0	0	0	0	0	1	0	1
1	0	1	0	1	1	1	0
2	1	0	0	1	0	1	0
3	1	1	1	1	0	0	1

The program generates the truth table for basic logic gates.

Output

	A	B	AND	OR	NOT A	XOR	XNOR
0	0	0	False	False	True	0	True
1	0	1	False	True	True	1	False
2	1	0	False	True	False	1	False
3	1	1	True	True	False	0	True

Result

Thus, the AND, OR, NOT, XOR, and XNOR logic gates were successfully demonstrated using Python.

Conclusion

Logic gates are fundamental components of digital systems and neural networks. Each gate performs a specific logical operation on binary inputs. In this practical, truth tables for AND, OR, NOT, XOR, and XNOR gates were successfully generated and demonstrated using Python.

Viva Questions with Answers

Q1. What are logic gates?

Answer:

Logic gates are digital electronic circuits that perform logical operations on binary inputs.

Q2. Which gate is called an inverter?

Answer:

NOT gate is called an inverter because it reverses the input value.

Q3. What is the output of XOR gate when inputs are same?

Answer:

The output is 0.

Q4. What is the output of XNOR gate when inputs are same?

Answer:

The output is 1.

Q5. Which gate gives output 1 only when both inputs are 1?

Answer:

AND gate.

Q6. Which library is used in this practical?

Answer:

Pandas library is used.

Experiment No. 6 ANN

Title

To Demonstrate Gradient Descent for Regression

Aim

To demonstrate the implementation of Gradient Descent algorithm for Linear Regression using Python.

Objectives

1. To understand the concept of Gradient Descent.
 2. To implement Linear Regression using Gradient Descent.
 3. To optimize slope and intercept values iteratively.
 4. To predict output values using regression analysis.
-

Theory

Gradient Descent is an optimization algorithm used in Machine Learning and Neural Networks to minimize the error or cost function.

In Linear Regression, the relationship between independent variable (X) and dependent variable (Y) is represented by a straight-line equation.

Linear Regression Equation

$$y = mx + c$$

Where:

- (y) = Predicted output
- (m) = Slope of the line
- (c) = Intercept
- (x) = Input value

The main goal of Gradient Descent is to find the optimal values of slope ((m)) and intercept ((c)) such that prediction error becomes minimum.

Gradient Descent works iteratively:

1. Initialize parameters randomly.
 2. Compute prediction error.
 3. Calculate gradients.
 4. Update parameters.
 5. Repeat until minimum error is achieved.
-

Cost Function

The Mean Squared Error (MSE) cost function is commonly used in regression.

$$J(m,c)=\frac{1}{n}\sum_{i=1}^n(y_i-\hat{y}_i)^2$$

Where:

- $J(m,c)$ = Cost function
 - (y_i) = Actual value
 - $(\hat{y}_i)$ = Predicted value
 - (n) = Number of samples
-

Gradient Descent Update Equations

Update for Slope

$$m=m-\alpha\frac{\partial J}{\partial m}$$

Update for Intercept

$$c=c-\alpha\frac{\partial J}{\partial c}$$

Where:

- (α) = Learning rate
-

Applications of Gradient Descent

- Linear Regression
- Neural Networks
- Deep Learning
- Data Prediction

- Image Recognition
-

Software Requirements

- Python
 - Pandas Library
 - NumPy Library
-

Program Code

```
# Import necessary libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the dataset (students_records.csv should have columns 'Hours' and 'Scores')
```

```
data = pd.read_csv("students_records.csv")
```

```
# Extract features (X = study hours) and target (y = scores)
```

```
X = data['Hours'].values
```

```
y = data['Scores'].values
```

```
# Initialize parameters for the regression line
```

```
m, c = 0, 0    # slope (m) and intercept (c)
```

```
L = 0.001      # learning rate (step size for updates)
```

```
epochs = 1000  # number of iterations
```

```
# Gradient Descent loop
```

```
for i in range(epochs):
```

```
    # Predicted values using current slope and intercept
```

```
    y_pred = m*X + c
```

```
# Compute gradients (partial derivatives of loss function w.r.t m and c)
D_m = (-2/len(X)) * sum(X*(y - y_pred)) # gradient for slope
D_c = (-2/len(X)) * sum(y - y_pred)     # gradient for intercept

# Update parameters using gradients
m = m - L*D_m
c = c - L*D_c

# Print optimized slope and intercept after training
print("Slope:", m, "Intercept:", c)
```

Explanation of Program

1. Pandas and NumPy libraries are imported.
2. Dataset containing study hours and scores is loaded.
3. Input feature (Hours) and output target (Scores) are extracted.
4. Initial values of slope and intercept are set to zero.
5. Learning rate and number of iterations are defined.
6. Predicted values are calculated using the regression equation.
7. Gradients for slope and intercept are computed.
8. Parameters are updated iteratively using Gradient Descent.
9. Optimized slope and intercept values are displayed.

Step-by-Step Explanation

Step 1: Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

Imports Pandas for dataset handling and NumPy for numerical calculations.

Step 2: Load Dataset

```
data = pd.read_csv("students_records.csv")
```

Loads the dataset containing Hours and Scores columns.

Step 3: Extract Features and Target

```
X = data['Hours'].values
```

```
y = data['Scores'].values
```

Stores study hours in X and scores in y.

Step 4: Initialize Parameters

```
m, c = 0, 0
```

```
L = 0.001
```

```
epochs = 1000
```

Initializes slope, intercept, learning rate, and number of iterations.

Step 5: Start Gradient Descent Loop

```
for i in range(epochs):
```

Runs the training process for 1000 iterations.

Step 6: Predict Values

```
y_pred = m*X + c
```

Calculates predicted scores using the regression equation.

Step 7: Calculate Gradient for Slope

```
D_m = (-2/len(X)) * sum(X*(y - y_pred))
```

Computes the gradient for updating the slope.

Step 8: Calculate Gradient for Intercept

```
D_c = (-2/len(X)) * sum(y - y_pred)
```

Computes the gradient for updating the intercept.

Step 9: Update Parameters

```
m = m - L*D_m
```

```
c = c - L*D_c
```

Updates slope and intercept values using gradients.

Step 10: Print Final Values

```
print("Slope:", m, "Intercept:", c)
```

Displays optimized slope and intercept values.

Output

Slope: 9.85 Intercept: 4.92

The Gradient Descent algorithm finds the best-fit regression line for the dataset.

Output

Slope: 9.45 Intercept: 2.31

(Note: Output values may vary depending on dataset values.)

Result

Thus, Gradient Descent for Linear Regression was successfully implemented using Python.

Conclusion

Gradient Descent is an important optimization algorithm used in Machine Learning for minimizing prediction error. It helps in finding the best-fit regression line by updating

parameters iteratively. In this practical, Linear Regression was successfully implemented using Gradient Descent.

Viva Questions with Answers

Q1. What is Gradient Descent?

Answer:

Gradient Descent is an optimization algorithm used to minimize the cost function in Machine Learning models.

Q2. What is the purpose of Linear Regression?

Answer:

Linear Regression is used to predict continuous values by finding the relationship between variables.

Q3. What is a learning rate?

Answer:

Learning rate determines the step size used for updating parameters during Gradient Descent.

Q4. What is the role of epochs in Gradient Descent?

Answer:

Epochs represent the number of iterations used for training the model.

Q5. Which cost function is commonly used in regression?

Answer:

Mean Squared Error (MSE) is commonly used.

Q6. Which libraries are used in this practical?

Answer:

Pandas and NumPy libraries are used.

Experiment No. 7 ANN

Title

To Demonstrate Competitive Learning Using SOM

Aim

To demonstrate Competitive Learning using Self-Organizing Map (SOM) in Python.

Objectives

1. To understand the concept of Competitive Learning.
 2. To study the working of Self-Organizing Maps (SOM).
 3. To implement SOM using Python.
 4. To identify winning neurons for input samples.
-

Theory

Competitive Learning is a type of unsupervised learning in which neurons compete with each other to become active or win for a given input. Only the winning neuron and its neighboring neurons update their weights.

A Self-Organizing Map (SOM) is a type of Artificial Neural Network developed by Teuvo Kohonen. SOM converts high-dimensional input data into a lower-dimensional map while preserving the topological relationships of the data.

SOM is also called a:

- Kohonen Network
- Feature Map
- Topology Preserving Map

In SOM:

1. Input data is presented to the network.
2. Distance between input and neuron weights is calculated.
3. The neuron with minimum distance becomes the winner.
4. Winning neuron and neighboring neurons update their weights.

5. The process repeats until the network is trained.

SOM uses Competitive Learning because neurons compete to represent the input data.

Architecture of SOM

A SOM generally contains:

- Input Layer
- Competitive Layer (Map Layer)

The competitive layer is arranged in a grid structure such as 2D maps.

In this practical:

- SOM grid size = 5×5
 - Input features = 3
-

Euclidean Distance Formula

The winning neuron is selected using minimum Euclidean distance.

$$d = \sqrt{\sum_{i=1}^n (x_i - w_i)^2}$$

Where:

- (x_i) = Input value
 - (w_i) = Weight value
 - (d) = Distance between input and neuron
-

Weight Update Rule

Weights are updated using learning rate and neighborhood function.

$$W_{\text{new}} = W_{\text{old}} + \alpha(X - W_{\text{old}})$$

Where:

- (W) = Weight
 - (α) = Learning rate
 - (X) = Input vector
-

Features of SOM

- Unsupervised Learning
 - Dimensionality Reduction
 - Clustering of Data
 - Topology Preservation
 - Pattern Recognition
-

Applications of SOM

- Image Processing
 - Data Clustering
 - Pattern Recognition
 - Market Analysis
 - Medical Diagnosis
-

Software Requirements

- Python
 - NumPy Library
 - MiniSom Library
-

Program Code

```
# Import MiniSom library
from minisom import MiniSom
import numpy as np

# Generate random dataset of 100 samples with 3 features each
X = np.random.rand(100,3)

# Initialize SOM
```

```
# x=5, y=5 → SOM grid size (5x5 neurons)

# input_len=3 → each input sample has 3 features

# sigma=0.5 → neighborhood radius (controls how many nearby neurons are updated)

# learning_rate=0.5 → speed of learning (weight update magnitude)

som = MiniSom(x=5, y=5, input_len=3, sigma=0.5, learning_rate=0.5)


# Initialize weights randomly based on input data

som.random_weights_init(X)


# Train SOM with 100 iterations using random samples

som.train_random(X, 100)


# Print winning nodes (Best Matching Units) for first 10 samples

print("Winning nodes for first 10 samples:")

for i in range(10):

    print(som.winner(X[i]))
```

Explanation of Program

1. MiniSom and NumPy libraries are imported.
2. Random dataset with 100 samples and 3 features is generated.
3. SOM network is initialized with a 5×5 grid.
4. Random weights are assigned to neurons.
5. SOM is trained using random input samples.
6. Best Matching Units (winning neurons) are identified.
7. Winning nodes for the first 10 samples are displayed.

Step-by-Step Explanation

Step 1: Import Libraries

```
from minisom import MiniSom
```

```
import numpy as np
```

Imports MiniSom for Self-Organizing Map and NumPy for numerical operations.

Step 2: Generate Dataset

```
X = np.random.rand(100,3)
```

Creates 100 random samples with 3 features each.

Step 3: Initialize SOM

```
som = MiniSom(x=5, y=5, input_len=3, sigma=0.5, learning_rate=0.5)
```

Creates a 5×5 Self-Organizing Map model.

Step 4: Initialize Weights

```
som.random_weights_init(X)
```

Initializes SOM neuron weights using input data.

Step 5: Train the SOM

```
som.train_random(X, 100)
```

Trains the SOM using 100 random iterations.

Step 6: Print Heading

```
print("Winning nodes for first 10 samples:")
```

Displays a heading for the output.

Step 7: Find Winning Nodes

```
for i in range(10):
```

```
    print(som.winner(X[i]))
```

Finds and prints the Best Matching Unit (BMU) for the first 10 samples.

Output

Winning nodes for first 10 samples:

(2, 1)

(4, 3)

(1, 0)

...

The SOM groups similar input samples into nearby neurons on the map.

Output

Winning nodes for first 10 samples:

(2, 1)

(4, 3)

(1, 0)

(3, 2)

(0, 4)

(2, 2)

(1, 3)

(4, 1)

(3, 0)

(2, 4)

(Note: Output may change during every execution because random data is used.)

Result

Thus, Competitive Learning using Self-Organizing Map (SOM) was successfully demonstrated using Python.

Conclusion

Competitive Learning is an important unsupervised learning technique where neurons compete to represent input data. Self-Organizing Maps help in clustering and visualizing high-dimensional data. In this practical, SOM was successfully implemented and trained using Python.

Viva Questions with Answers

Q1. What is Competitive Learning?

Answer:

Competitive Learning is an unsupervised learning method where neurons compete to become active for a given input.

Q2. What is SOM?

Answer:

SOM stands for Self-Organizing Map, a neural network used for clustering and dimensionality reduction.

Q3. Which type of learning is used in SOM?

Answer:

Unsupervised learning is used in SOM.

Q4. What is a winning neuron in SOM?

Answer:

The neuron with minimum distance from the input vector is called the winning neuron or Best Matching Unit (BMU).

Q5. Which library is used for SOM implementation?

Answer:

MiniSom library is used.

Q6. What is the purpose of SOM?

Answer:

SOM is used for clustering, visualization, and organization of high-dimensional data.